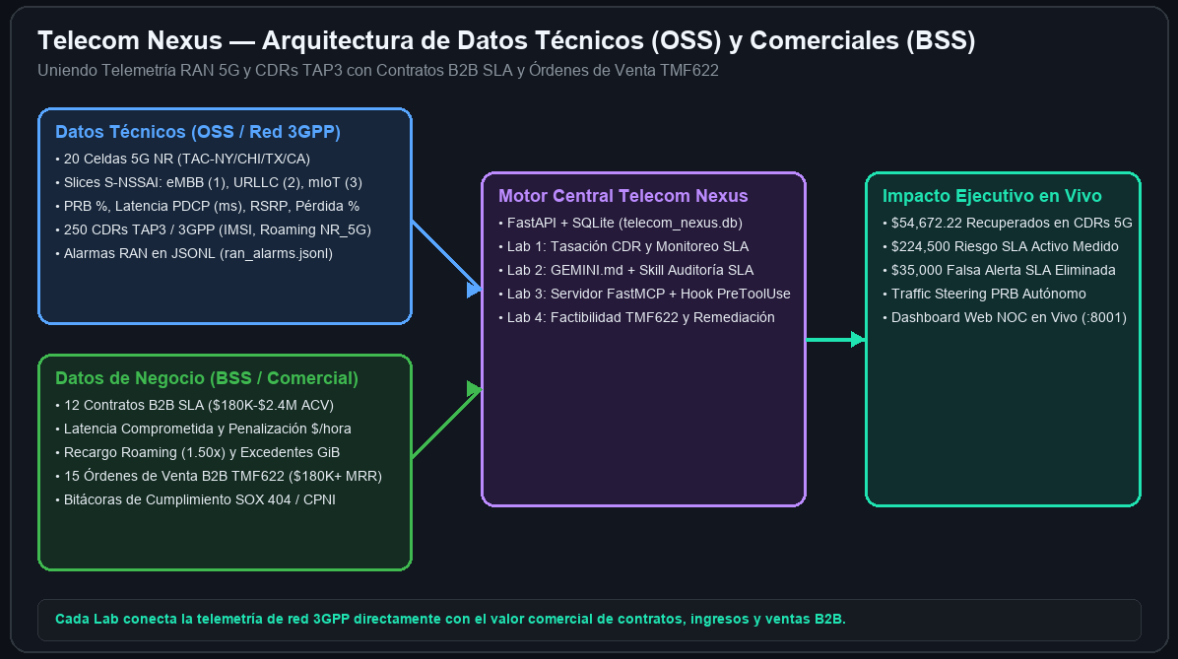
Ingeniería Agéntica con Google Antigravity — Edición Telecom: Construyendo 'Telecom Nexus'

SLA de Slices 5G, Aseguramiento de Ingresos CDR e Inteligencia de Contratos B2B (Impartido por Tryolabs)



Misión del Taller y Arquitectura de Datos

- **Datos Técnicos OSS:** 20 celdas 5G NR (gNodeB, TAC, S-NSSAI SST 1/2/3, PRB%, latencia PDCP)
- **CDRs TAP3 / 3GPP:** 250 registros de voz/datos (IMSI, MSISDN, roaming NR_5G, volumen GiB)
- **Datos de Negocio BSS:** 12 Contratos B2B SLA Empresariales ($180K–$2.4M ACV, penalizaciones $/h)
- **Pipeline de Ventas TMF622:** 15 órdenes B2B pendientes + telemetría de alarmas RAN en JSONL
- **Impacto Medible:** Recuperar $54,672.22 en CDRs no facturados y gestionar $224,500 en riesgo SLA activo

Ruta Práctica de 4 Horas: Del Triage a Sistemas Multi-Agente ADK y A2A

Cuatro cápsulas teóricas de 6 minutos seguidas de cuatro laboratorios progresivos de 52–55 minutos



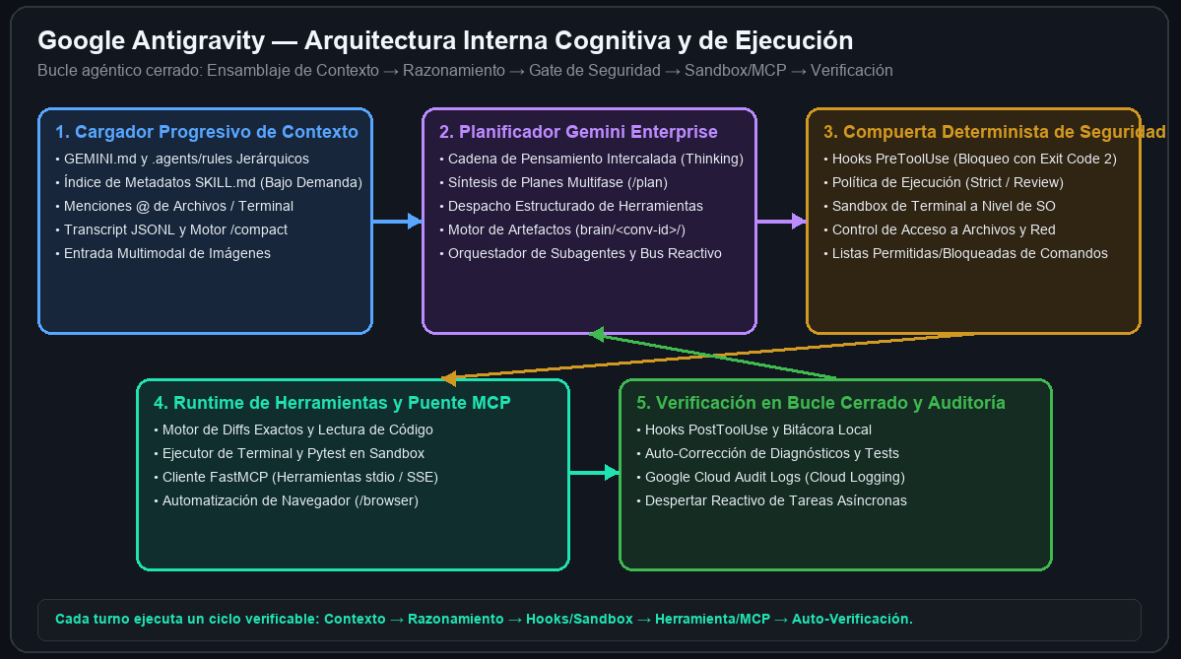
Cronograma Minuto a Minuto

- **Hora 1 (00:00–00:58):** [6m Teoría] Arquitectura Interna y Superficies → [52m Lab 1] Triage y Recuperación Financiera
- **Hora 2 (00:58–01:58):** [6m Teoría] Rules y Skills Progresivos → [54m Lab 2] Estándares de Dominio y Skill Ejecutivo
- **Hora 3 (01:58–02:59):** [6m Teoría] MCP, Hooks y Gobernanza Cloud → [55m Lab 3] Servidor FastMCP Local y Hooks
- **Hora 4 (02:59–04:00):** [6m Teoría] Subagentes, ADK, A2A y Terraform → [55m Lab 4] Capstone Multi-Agente, ADK Dev UI y Cloud
- **Checkpoints Auto-Recuperables:** ¡Ejecutar ./labs/lab-0X/install.sh aplica las soluciones previas automáticamente!

Cada laboratorio incluye LAB_0X_INSTRUCTIONS_ES.pdf, README.es.md, PROMPTS.es.md, install.sh y verify.sh.

Cómo Funciona Google Antigravity Internamente

Dentro del planificador cognitivo, ensamblador de contexto, compuerta de seguridad y runtime en sandbox



El Ciclo Interno de Ejecución en 5 Etapas

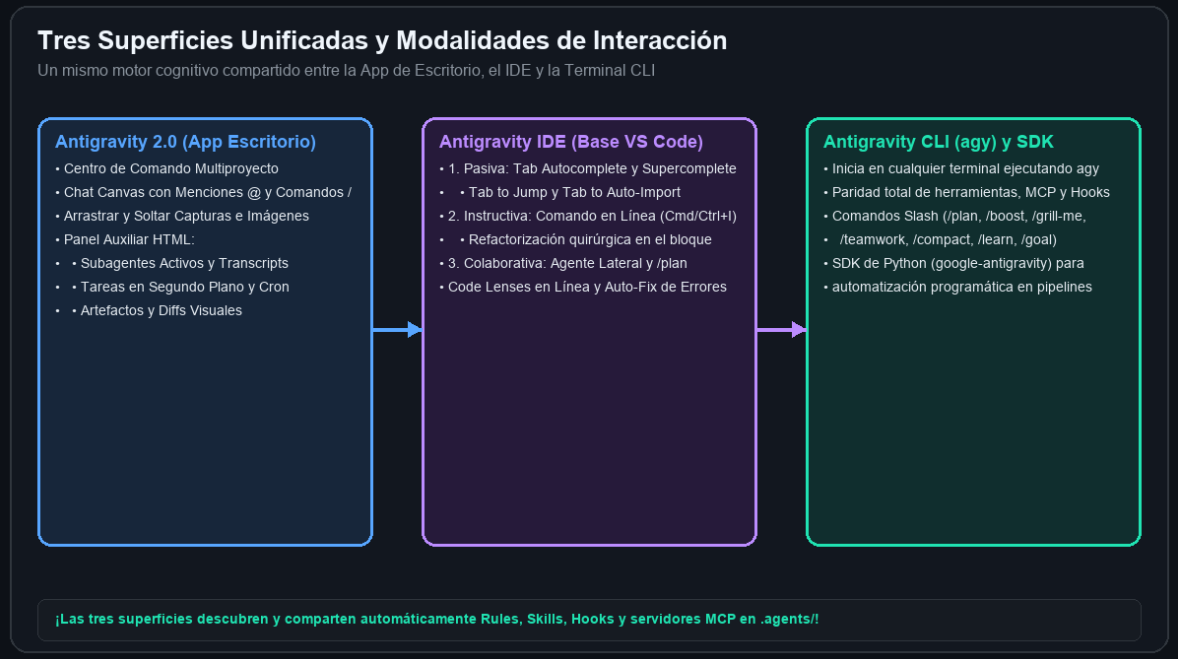
- **1. Cargador Progresivo de Contexto:** Recorre el árbol de directorios buscando GEMINI.md y .agents/rules, indexa metadatos YAML de SKILL.md, resuelve menciones @ y gestiona transcript.jsonl
- **2. Planificador Gemini Enterprise:** Genera razonamiento intercalado (thinking), sintetiza planes (/plan) y despacha herramientas o subagentes en paralelo
- **3. Compuerta Determinista de Seguridad:** Evalúa hooks PreToolUse (código 2 bloquea la acción), políticas de ejecución, ACLs y sandbox de terminal
- **4. Runtime de Herramientas y Puente MCP:** Aplica diffs exactos, ejecuta pytest en sandbox e invoca herramientas FastMCP por stdio/SSE
- **5. Verificación en Bucle Cerrado:** Ejecuta hooks PostToolUse, analiza resultados de tests y se autocorrige hasta pasar todas las pruebas

Concepto Clave: Antigravity no es solo un chat, sino un motor de ingeniería en bucle cerrado verificable. | **Slide 3/15**

tryo•labs

Antigravity 2.0, IDE y CLI (`agy`) — Modalidades Clave

Seleccionando la modalidad y comando slash ideal para cada tarea de ingeniería



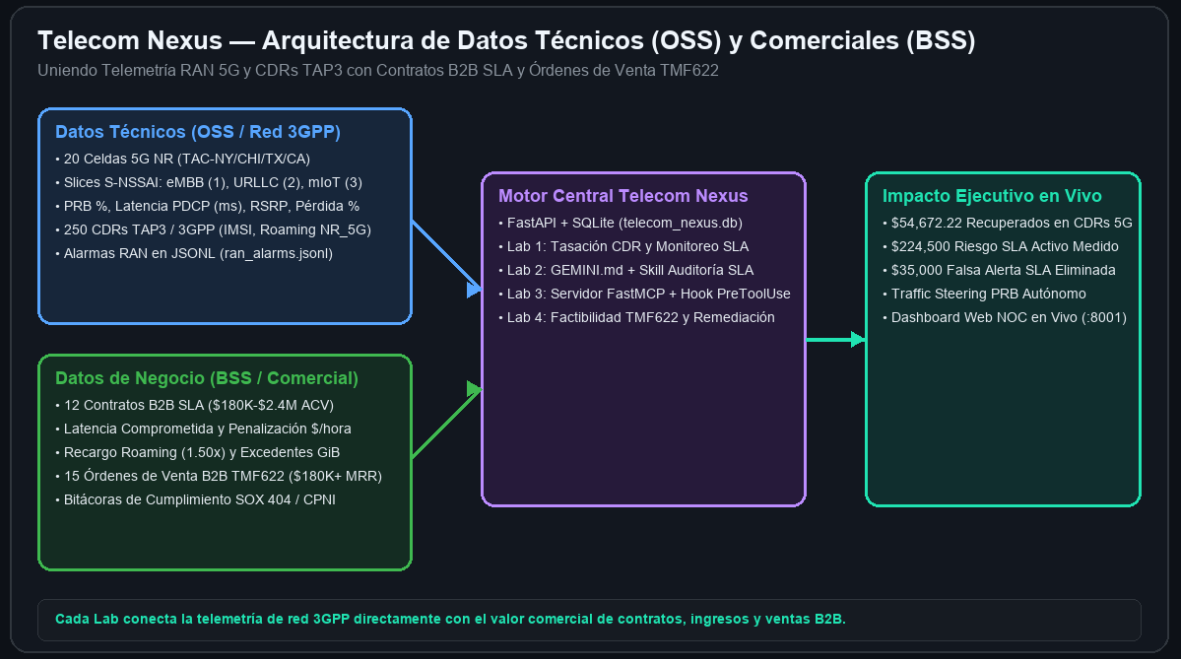
Funcionalidades que Explorarás Hoy

- **Modalidad Pasiva (IDE Tab):** Supercomplete sensible al contexto, Tab-to-Jump entre archivos y Tab-to-Auto-Import
- **Modalidad Instructiva (Cmd/Ctrl+I y Code Lenses):** Selecciona un bloque para refactorización quirúrgica o haz clic en 'Write Tests'
- **Modalidad Colaborativa (Chat Canvas y CLI `agy`):** Razonamiento multi-archivo con @archivo, @carpeta, @terminal y capturas de pantalla
- **Comandos Slash de Alto Impacto:** /plan (arquitectura paso a paso), /grill-me (entrevista de diseño), /boost, /teamwork, /learn y /compact
- **Panel Auxiliar HTML:** Inspección en vivo de Subagentes, Tareas en Segundo Plano, Artefactos y Diffs de Archivos

Tip Pro: Inicia tareas complejas con /plan y menciones @archivo; usa Cmd+I para ajustes puntuales de fórmulas.

Lab 1: Triage de SLA 5G y Facturación CDR — Objetivos y Arquitectura

52 Minutos • Guía PDF Imprimible: labs/lab-01-revenue-sla-triage/LAB_01_INSTRUCTIONS_ES.pdf



Objetivos del Lab 1 y Qué Construirás

- **Objetivo 1 (Triage Agéntico Multi-Archivo):** Dominar `@menciones`, `/plan` y edición en línea `Cmd/Ctrl+I` en Antigravity correlacionando alarmas RAN (`ran_alarms.jsonl`) con código BSS.
- **Objetivo 2 (Aseguramiento de Ingresos y SLA):** Recuperar `$54,672.22 USD` en CDRs 5G subfacturados y eliminar `$35,000 USD` en falsas alertas SLA sobre contratos no activos.
- **Qué Construirás #1 (Motor de Tasación TAP3 3GPP):** Refactorizarás `src/services/cdr_rating.py` imponiendo GiB binario (`Decimal(1024 ** 3)`) y recargo de roaming en toda red visitada (`NR_5G` y `EUTRAN_4G`).
- **Qué Construirás #2 (Monitor de SLA B2B):** Refactorizarás `src/services/sla_monitor.py` filtrando `WHERE status = 'ACTIVE'` y evaluando `worst_latency > committed_latency`.
- **Entregable y Verificación:** Pasar los 4 tests en `tests/test_lab01_triage.py` (`./labs/lab-01-revenue-sla-triage/verify.sh`) y validar KPIs en `http://localhost:8001`.

tryo•labs

Instrucciones del Lab 1 y Prompts para Copiar y Pegar

Sigue los Pasos 1–5 a la izquierda y copia los prompts desde labs/lab-01-revenue-sla-triage/PROMPTS.es.md

Instrucciones Paso a Paso del Lab 1

- **Paso 1 (Inicializar Lab 01):** Ejecuta `./labs/lab-01-revenue-sla-triage/install.sh` para poblar `data/telecom_nexus.db`.
- **Paso 2 (Reproducir Fallos):** Ejecuta `./.venv/bin/pytest tests/test_lab01_triage.py` para observar los fallos de CDR y SLA.
- **Paso 3 (Corregir `src/services/cdr_rating.py`):** Usa `BYTES_DIVISOR = Decimal(1024 ** 3)` (GiB binario según 3GPP TS 32.298) y aplica `roaming_markup` cuando `roaming_plmn != HOME_PLMN`.
- **Paso 4 (Corregir `src/services/sla_monitor.py`):** Filtra `WHERE status = 'ACTIVE'` y cambia `latency_breach` a `worst_latency > committed_latency`.
- **Paso 5 (Verificar):** Ejecuta `./labs/lab-01-revenue-sla-triage/verify.sh` (`4 passed`: `$54,672.22` recuperados; 5 contratos `ACTIVE` en alerta = `$224,500.00`).

Prompts Optimizados (PROMPTS.es.md) y Verificación

- **Prompt 1 — Diagnóstico (@Menciones):** '@data/logs/ran_alarms.jsonl @src/services/cdr_rating.py @src/services/sla_monitor.py @tests/test_lab01_triage.py Ejecuta pytest para reproducir ALM-BSS-8006 y ALM-3GPP-8001..8005 y explica la causa raíz exacta.'
- **Prompt 2 — Planificación (/plan):** '/plan Crea un plan paso a paso para corregir cdr_rating.py (GiB 10243 y roaming NR_5G) y sla_monitor.py (contratos ACTIVE y worst_latency > committed_latency).'
- **Prompt 3 — Edición en Línea (Cmd+I):** Selecciona líneas 19–48 en `cdr_rating.py` -> 'Usa Decimal(10243) y aplica recargo roaming a todo visiting_plmn != HOME_PLMN incluyendo NR_5G.'
- **Prompt 4 — Verificación Autónoma:** 'Aplica las correcciones en @src/services/cdr_rating.py y @src/services/sla_monitor.py y ejecuta ./labs/lab-01-revenue-sla-triage/verify.sh.'

tryo•labs

Cómo Funcionan Internamente las Rules y Skills Progresivos

Especializando a Antigravity en tu industria sin saturar la ventana de contexto



Rules vs. Skills vs. /learn

- **Reglas Jerárquicas (GEMINI.md y .agents/rules/*.md):** Invariantes de dominio cargados desde la raíz del workspace hasta los subdirectorios (deduplicados automáticamente)
- **Skills con Divulgación Progresiva (.agents/skills/<name>/SKILL.md):** Solo el frontmatter YAML (name, description) entra al prompt inicial; el cuerpo completo y scripts/ se cargan SOLO al activarse
- **Memoria Auto-Mejorable (/learn):** Pide a Antigravity codificar un patrón aprendido durante el debugging en una regla o skill permanente
- **Jerarquía de Prioridad:** Workspace (.agents/) > JSON Declarado (skills.json) > Global (~/.gemini/config/) > Integrado

Regla Práctica: Coloca invariantes permanentes en GEMINI.md/rules y flujos multifase con scripts en SKILL.md.

tryo•labs

Lab 2: Rules de Workspace y Skills Progresivos — Objetivos y Diseño

54 Minutos • Guía PDF Imprimible: labs/lab-02-rules-and-skills/LAB_02_INSTRUCTIONS_ES.pdf



Objetivos del Lab 2 y Qué Construirás

- **Objetivo 1 (Reglas Jerárquicas de Dominio):** Codificar invariantes permanentes 3GPP, TAP3, TMF622 y privacidad CPNI para que Antigravity las respete siempre.
- **Objetivo 2 (Skills con Divulgación Progresiva):** Empaquetar una auditoría ejecutiva en `.agents/skills/` cargando solo el frontmatter YAML hasta su activación.
- **Qué Construirás #1 (`GEMINI.md` y `.agents/rules/telecom-standards.md`):** Memoria y reglas del workspace exigiendo `Decimal(ROUND_HALF_UP)`, GiB binario (`1024**3`), `PRB < 80%` y máscara CPNI `IMSI`/`MSISDN`.
- **Qué Construirás #2 (`.agents/skills/sla-revenue-audit/SKILL.md`):** Skill progresivo (`name: sla-revenue-audit`) que ejecuta `scripts/run_sla_audit.py`.
- **Entregable y Verificación:** Generar `data/exports/sla_revenue_audit_report.json` y `.md` y pasar los 4 tests de `./labs/lab-02-rules-and-skills/verify.sh`.

tryo•labs

Instrucciones del Lab 2 y Prompts para Copiar y Pegar

Sigue los Pasos 1–5 a la izquierda y copia los prompts desde labs/lab-02-rules-and-skills/PROMPTS.es.md

Instrucciones Paso a Paso del Lab 2

- **Paso 1 (Inicializar Lab 02):** Ejecuta `./labs/lab-02-rules-and-skills/install.sh` (aplica Lab 01 automáticamente).
- **Paso 2 (Crear `GEMINI.md` en la raíz):** Define tablas OSS/BSS (`ran_telemetry`, `b2b_contracts`, `cdr_records`), `Decimal(ROUND_HALF_UP)` y GiB binario (`1024 ** 3`).
- **Paso 3 (Crear `.agents/rules/telecom-standards.md`):** Codifica estándares 3GPP TS 28.552 (`PRB < 80%`), roaming TAP3 `NR_5G`, estados TMF622 y enmascaramiento CPNI de `IMSI`/`MSISDN`.
- **Paso 4 (Crear `.agents/skills/sla-revenue-audit/SKILL.md`):** Incluye frontmatter YAML (`name: sla-revenue-audit`, `description: ...`) e instrucciones para correr `./.venv/bin/python scripts/run_sla_audit.py`.
- **Paso 5 (Verificar):** Ejecuta `./labs/lab-02-rules-and-skills/verify.sh` (`4 passed` + reportes JSON y Markdown).

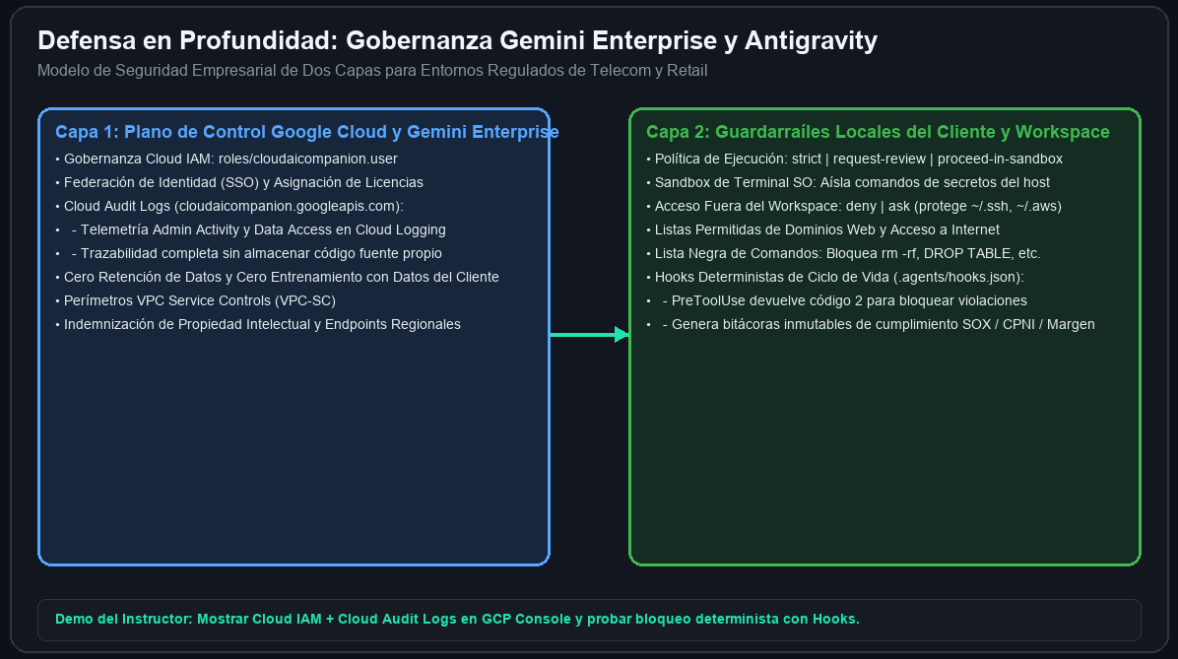
Prompts Optimizados (PROMPTS.es.md) y Verificación

- **Prompt 1 — Crear Rules de Dominio:** '/plan Crea @GEMINI.md y @.agents/rules/telecom-standards.md codificando umbrales 3GPP TS 28.552, GiB binario (10243), Decimal ROUND_HALF_UP, estados TMF622 y privacidad CPNI.'
- **Prompt 2 — Crear Skill Progresivo:** 'Crea @.agents/skills/sla-revenue-audit/SKILL.md con frontmatter YAML (name: sla-revenue-audit) que ejecute ./scripts/run_sla_audit.py y genere data/exports/sla_revenue_audit_report.json y .md.'
- **Prompt 3 — Invocar el Skill:** 'Ejecuta una auditoría ejecutiva de SLA de Slices 5G y aseguramiento de ingresos CDR usando nuestro skill del workspace.'
- **Comando de Verificación:** Ejecuta `./labs/lab-02-rules-and-skills/verify.sh` y prueba `/learn`.

tryo•labs

Puente MCP en Vivo, Hooks Deterministas y Gobernanza Gemini Enterprise

Conectando bases de datos en tiempo real con seguridad empresarial demostrable en Google Cloud



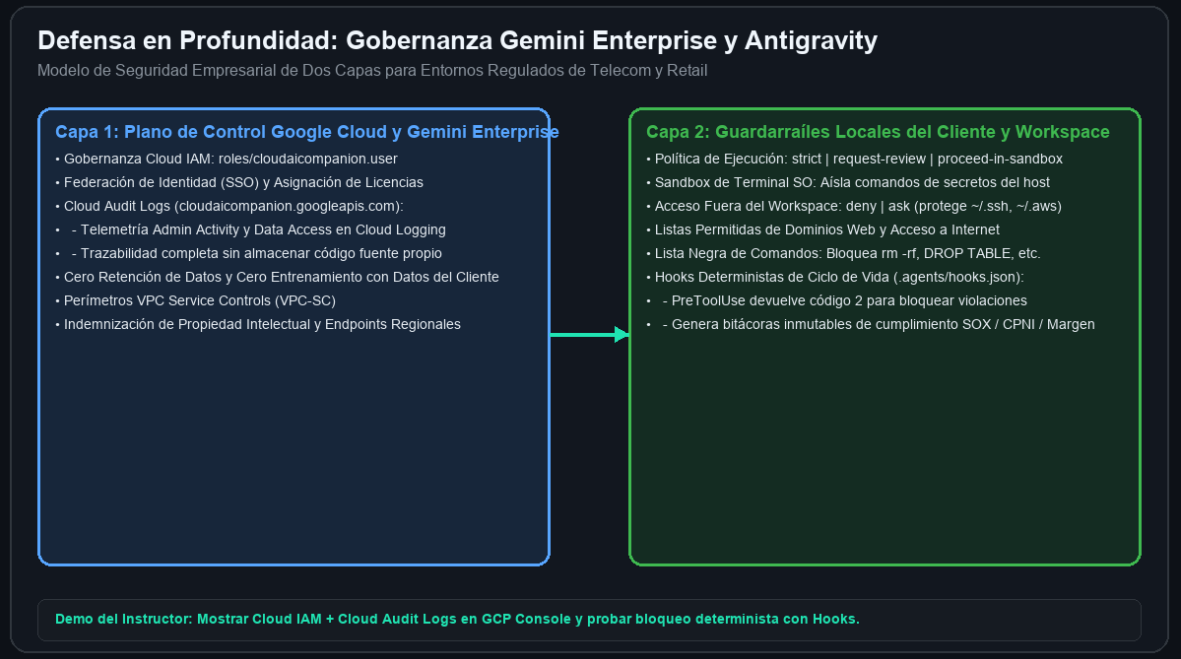
Seguridad y Gobernanza en 2 Capas

- **Capa 1 — Google Cloud y Gemini Enterprise (Demo en Vivo del Instructor):** Cloud IAM (roles/cloudaicompanion.user), Cloud Audit Logs (cloudaicompanion.googleapis.com), perímetros VPC-SC, Cero Retención y Cero Entrenamiento con código del cliente
- **Capa 2 — Seguridad Local en Antigravity:** Política de Ejecución (strict / request-review / proceed-in-sandbox), Sandbox de Terminal SO, bloqueo de archivos fuera del workspace y listas permitidas/bloqueadas
- **Contexto en Vivo vía FastMCP (.agents/mcp_config.json):** Expone herramientas Python tipadas sobre SQLite y telemetría vía stdio/SSE
- **Hooks Deterministas (.agents/hooks.json):** El script PreToolUse inspecciona el JSON en stdin; devolver código 2 bloquea violaciones e imprime un registro de auditoría inmutable

Demo del Instructor: Mostrar Cloud Audit Logs en GCP Console + disparar un bloqueo PreToolUse con código 2 localmente. | **Slide 10/15**

Lab 3: Servidor FastMCP y Hooks Deterministas — Objetivos y Diseño

55 Minutos • Guía PDF Imprimible: labs/lab-03-mcp-and-hooks/LAB_03_INSTRUCTIONS_ES.pdf



Objetivos del Lab 3 y Qué Construirás

- **Objetivo 1 (Conexión a Datos en Vivo vía MCP):** Exponer herramientas SQLite OSS/BSS tipadas a Antigravity mediante un servidor `FastMCP`.
- **Objetivo 2 (Seguridad Determinista y Privacidad CPNI):** Enmascarar `IMSI`/`MSISDN` (`3104***0001`) y bloquear SQL destructivo (`DROP`/`DELETE`) con código de salida `2` mediante hooks `PreToolUse`.
- **Qué Construirás #1 (`mcp_server/telecom_mcp.py` y `.agents/mcp_config.json`):** 4 herramientas `@mcp.tool()` (`query_ran_cell_health`, `calculate_contract_penalty_exposure`, `inspect_unbilled_cdrs`, `evaluate_slice_capacity`).
- **Qué Construirás #2 (`.agents/hooks.json` y `scripts/audit_hook.py`):** Compuerta determinista `PreToolUse` y `PostToolUse` que bloquea violaciones con código `2` y audita en `data/logs/hook_audit.log`.
- **Entregable y Verificación:** Pasar los 3 tests de MCP y Hooks (`11 passed` acumulados) en `./labs/lab-03-mcp-and-hooks/verify.sh`.

tryo•labs

Instrucciones del Lab 3 y Prompts para Copiar y Pegar

Sigue los Pasos 1–5 a la izquierda y copia los prompts desde labs/lab-03-mcp-and-hooks/PROMPTS.es.md

Instrucciones Paso a Paso del Lab 3

- **Paso 1 (Inicializar Lab 03):** Ejecuta `./labs/lab-03-mcp-and-hooks/install.sh` (aplica Labs 01–02 automáticamente).
- **Paso 2 (Completar `mcp_server/telecom_mcp.py`):** Implementa `calculate_contract_penalty_exposure`, `inspect_unbilled_cdrs` (enmascarando `imsi`/`msisdn` con `_mask_cpni`) y `evaluate_slice_capacity`.
- **Paso 3 (Registrar `.agents/mcp_config.json`):** Configura `telecom-nexus-mcp` invocando `./.venv/bin/python mcp_server/telecom_mcp.py`.
- **Paso 4 (Configurar `.agents/hooks.json`):** Conecta `PreToolUse` y `PostToolUse` a `scripts/audit_hook.py` (bloquea `DROP`/`TRUNCATE`/`DELETE` con código `2`).
- **Paso 5 (Verificar):** Ejecuta `./labs/lab-03-mcp-and-hooks/verify.sh` (`11 passed` acumulados en Labs 01–03 + bitácora de auditoría).

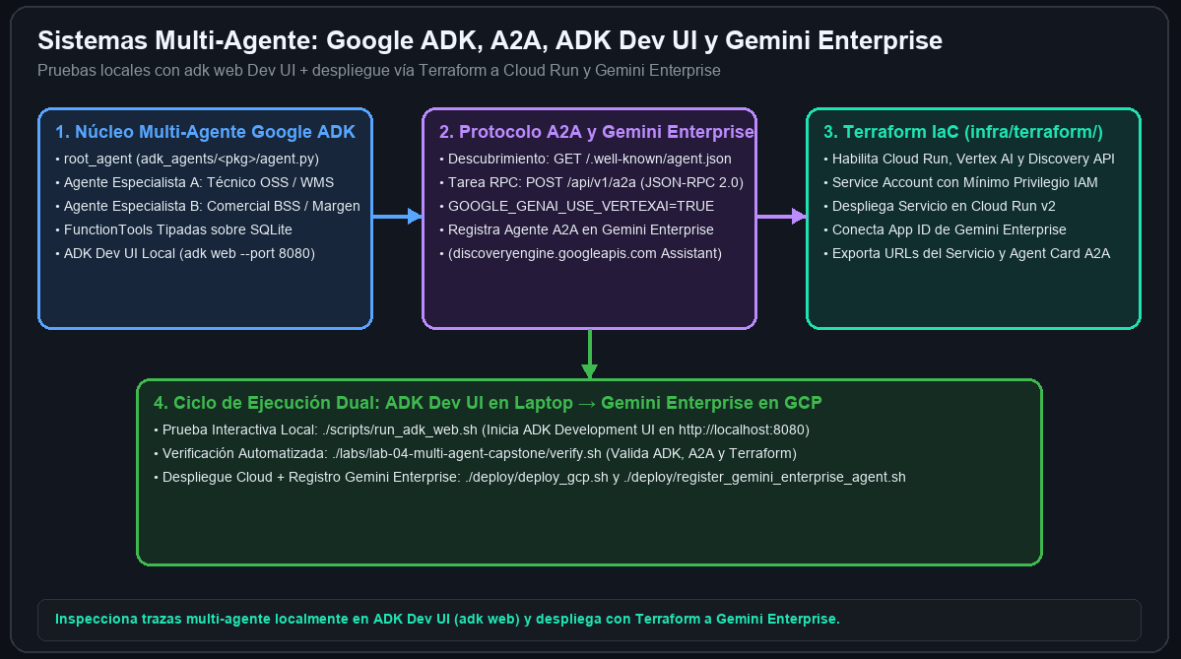
Prompts Optimizados (PROMPTS.es.md) y Verificación

- **Prompt 1 — Completar Servidor FastMCP:** '/plan Implementa las 3 funciones @mcp.tool() restantes en @mcp_server/telecom_mcp.py (calculate_contract_penalty_exposure, inspect_unbilled_cdrs con máscara CPNI y evaluate_slice_capacity) según @tests/test_lab03_mcp_hooks.py.'
- **Prompt 2 — Configurar MCP y Hooks:** 'Crea @.agents/mcp_config.json para telecom-nexus-mcp y @.agents/hooks.json conectando PreToolUse y PostToolUse a ./scripts/audit_hook.py.'
- **Prompt 3 — Probar Bloqueo del Hook (Código 2):** Ejecuta `echo '{"command":"sqlite3 data/telecom_nexus.db \"DROP TABLE b2b_contracts;\""}' | ./.venv/bin/python scripts/audit_hook.py --phase pre`
- **Comando de Verificación:** Ejecuta `./labs/lab-03-mcp-and-hooks/verify.sh` y revisa `data/logs/hook_audit.log`.

tryo•labs

Sistemas Multi-Agente con Google ADK, Protocolo A2A y Terraform

Diseñando arquitecturas multi-agente localmente con `adk web` y desplegando a Cloud Run y Gemini Enterprise

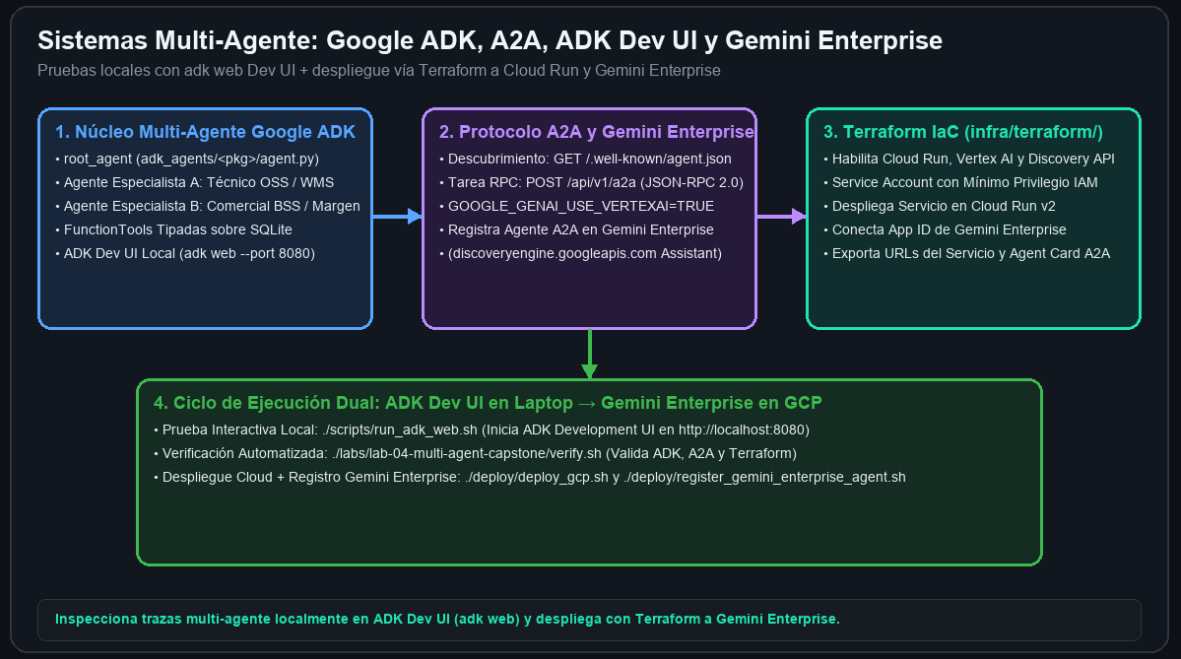


Arquitectura Multi-Agente de Punta a Punta

- **Subagentes de Antigravity (/teamwork, /boost, /grill-me):** Delega tareas paralelas de investigación y programación dentro de Antigravity
- **Google ADK (Agent Development Kit):** Construye jerarquías multi-agente de dominio (`root_agent` coordinando agentes especialistas Técnico y Comercial con herramientas tipadas)
- **Pruebas Locales con ADK Dev UI (`adk web`):** Ejecuta `./scripts/run_adk_web.sh` (http://localhost:8080) para inspeccionar trazas de eventos multi-agente, llamadas a herramientas y estado
- **Protocolo A2A (Agent2Agent):** Publica `GET /.well-known/agent.json` (Agent Card) y `POST /api/v1/a2a` (JSON-RPC 2.0 `tasks/send`) para interoperabilidad entre agentes
- **Terraform + Gemini Enterprise:** Aprovisiona Cloud Run v2 e IAM vía `infra/terraform/` (`deploy_gcp.sh`) y registra tu agente A2A en Gemini Enterprise (`register_gemini_enterprise_agent.sh`)

Lab 4 Capstone: Sistemas Multi-Agente ADK, A2A y Terraform — Objetivos

55 Minutos • Guía PDF Imprimible: labs/lab-04-multi-agent-capstone/LAB_04_INSTRUCTIONS_ES.pdf



Objetivos del Lab 4 y Qué Construirás

- **Objetivo 1 (Diseño Multi-Agente con Google ADK y A2A):** Construir una jerarquía coordinador + especialistas usando Google ADK (`LlmAgent`) y exponerla mediante el protocolo A2A JSON-RPC 2.0.
- **Objetivo 2 (Prueba Local en `adk web` y Despliegue Cloud con Terraform):** Probar trazas multi-agente en `adk web` (`http://localhost:8080`) y aprovisionar Cloud Run + Gemini Enterprise vía Terraform.
- **Qué Construirás #1 (`src/services/order_feasibility.py`):** Evaluador de factibilidad TMF622 (`PRB < 80%`) y motor autónomo de balanceo hacia celdas vecinas (`remediate_slice_for_order`).
- Qué Construirás #2 (`src/agents/adk_a2a_orchestrator.py` y `adk_agents/telecom_nexus/`): `RANSliceCapacityAgent` + `BSSCommercialSLAAgent` coordinados por `TelecomNexusOrchestratorAgent`, más `GET /.well-known/agent.json` y `POST /api/v1/a2a`.
- **Qué Construirás #3 (Infraestructura Cloud):** Validar `infra/terraform/main.tf` y el registro en Gemini Enterprise con `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run`.

tryo•labs

Instrucciones del Lab 4, Prueba en `adk web` y Despliegue Terraform

Sigue los Pasos 1–5 a la izquierda y copia los prompts desde labs/lab-04-multi-agent-capstone/PROMPTS.es.md

Instrucciones Paso a Paso del Lab 4

- **Paso 1 (Inicializar Lab 04):** Ejecuta `./labs/lab-04-multi-agent-capstone/install.sh`.
- **Paso 2 (Motor de Factibilidad TMF622):** Implementa `evaluate_order_feasibility()` y `remediate_slice_for_order()` en `src/services/order_feasibility.py` (balanceo PRB hacia celdas vecinas).
- **Paso 3 (Orquestador Google ADK y A2A):** Completa `src/agents/adk_a2a_orchestrator.py` (`RANSliceCapacityAgent`, `BSSCommercialSLAAgent`, `TelecomNexusOrchestratorAgent`, `GET /.well-known/agent.json`, `POST /api/v1/a2a`).
- **Paso 4 (Interfaz Local `adk web`):** Ejecuta `./scripts/run_adk_web.sh` y prueba `telecom_nexus` en `http://localhost:8080`.
- **Paso 5 (Terraform y Verificación Total):** Ejecuta `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run` y `./labs/lab-04-multi-agent-capstone/verify.sh` (**¡21/21 tests pasando!**).

Prompts Optimizados, `adk web` y Comandos Cloud

- **Prompt 1 — Motor de Factibilidad:** '/plan Implementa evaluate_order_feasibility y remediate_slice_for_order en @src/services/order_feasibility.py según 3GPP TS 28.552 PRB < 80% y desvío a celda vecina.'
- **Prompt 2 — Orquestador ADK + A2A:** '/teamwork Implementa RANSliceCapacityAgent, BSSCommercialSLAAgent, TelecomNexusOrchestratorAgent, get_a2a_agent_card y handle_a2a_jsonrpc_request en @src/agents/adk_a2a_orchestrator.py.'
- **Prompt 3 — Interactivo en `adk web` (http://localhost:8080):** 'Evalúa la orden ORD-TMF622-9001, ejecuta remediación autónoma de slice 5G y calcula el ACV protegido.'
- **Despliegue y Verificación:** Ejecuta `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run` y `./labs/lab-04-multi-agent-capstone/verify.sh`.